

Solution QUIZ 3 (Dated: 29-10-2024)
Data Structures and Algorithm Analysis

Roll No: _____

Name: _____

Q1. Consider a Job Scheduler that selects a job for processing in every iteration. In each iteration, it adds the currently running job to the tail of the queue and selects the next job at the front. If the job has waited at least 5 seconds in the queue, the scheduler selects it for processing, printing the message: "Job k is selected". Otherwise, the job is moved to the end of the queue, and the scheduler prints: "Job k sent back". Write code to simulate this selection process, printing the appropriate messages in each iteration.

Note: The queue itself does not need to be created, as it is assumed available as a class member and always contains enough jobs to select next job. The jobs are infinitely long, therefore, run an infinite loop.

```
//consider queue<pair<int, int>> where pair has job id and time
while (true) {
    //assume currentTime is a function that returns the current system time
    q.push({cJobId++, currentTime()});
    while (true) {
        pair<int, int> job = q.front();
        jobId = job.first, jTime = job.second;
        if (currentTime() - jTime < 5) {
            cout << "Job " << jobId << " sent back\n";
            q.push({jobId, jTime});
            q.pop();
        } else {
            cout << "Job " << jobId << " is selected\n";
            q.pop();
            break;
        }
    }
}
```

Q2. A post-fix expression has single digit integers only. Using stack evaluate the post-fix expression.

```
stack<int> st;
for (char c : expression){
    if (c >= '0' && c <= '9')
        st.push(c - '0');          '0' is subtracted to make character to digit
    else {
        n2 = st.front(), st.pop();
        n1 = st.front(), st.pop();
        if (c == '+')      st.push(n1 + n2);
        else if (c == '-') st.push(n1 - n2);
        else if (c == '*') st.push(n1 * n2);
        else if (c == '/') st.push(n1 / n2);
        else               st.push(n1 % n2);
    }
}
cout << st.top() ;           stack top has result at the end
```

Q3. Write member function for Linked List class to compare two linked lists. If both lists have equal sum return 0, if the first list has larger sum return 1, otherwise return -1.

```
int compare(LinkedList& list){
    Node *t1 = head, *t2 = list.head;
    int sum1 = 0, sum2 = 0;
    for ( ; t1 ; t1 = t1 -> next)    sum1 += t1 -> val;
    for ( ; t2 ; t2 = t2 -> next)    sum2 += t2 -> val;
    if (sum1 == sum2) return 0;
    else if (sum1 > sum2) return 1;
    return -1;
}
```

Q4. Write member function for Linked List class to reverse the linked list without modifying the data/ value of nodes.

```
stack<int> st;
for (t = head ; t ; t = t -> next)
    st.push(t->val);
for (t = head ; t ; t = t -> next){
    t -> val = st.top();
    st.pop();
}
```